چالش ۳۰ روزهٔ اندروید

آمادگی مصاحبهٔ اندروید — جونیور ← مید ← سینیور ← آرکیتکت

نسخهٔ ۲۰۲۶

روز ۲۲

سیستم build و Gradle — بخش ۲: وابستگی‌ها، variantها، R8 و build در مقیاس بزرگ



اسماعیل مرادی

@esmaeeil-moradi700

چرا وابستگی‌ها و variantها در ۲۰۲۶ مهم‌اند؟

روز ۲۱ build را سریع و مرتب کرد. بخش ۲ دربارهٔ **درست نگه داشتن آن در مقیاس بزرگ** است — همان دسته باگ‌هایی که در debug رد می‌شوند و در release کرش می‌کنند.

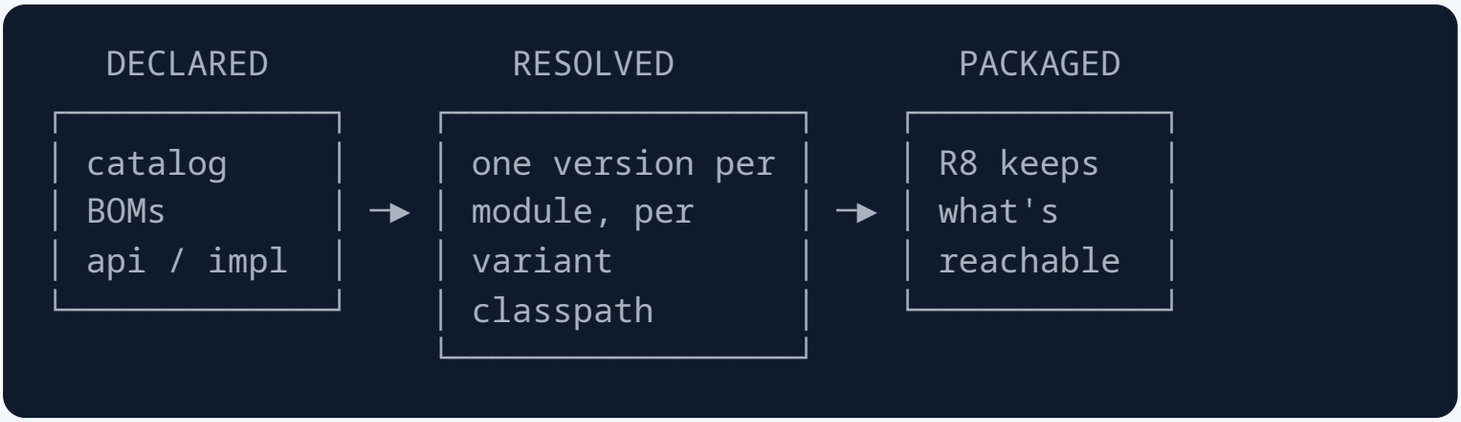
- گراف وابستگی‌ها تعیین می‌کند چه چیزی دوباره کامپایل شود: `api` همه‌جا، یک تغییر را به rebuild نیمی از اپ تبدیل می‌کند
- Gradle برای هر ماژول روی هر classpath **یک نسخه** انتخاب می‌کند — اغلب نه همانی که شما نوشته‌اید
- R8 فقط روی buildهای release اجرا می‌شود؛ یک keep rule جاافتاده یعنی کرش در production. AGP 9 قواعد keep را سخت‌گیرانه‌تر کرد
- AGP 9 نوع‌های قدیمی DSL و variant API قدیمی را به‌طور پیش‌فرض پنهان می‌کند — ارتقا استراتژی می‌خواهد
- taskهای سفارشی باید lazy و cacheable باشند، وگرنه هر چه در بخش ۱ به دست آمد را خراب می‌کنند

سطح	سؤال‌ها
جونیور	مقایسهٔ `api` و `implementation`، build type و flavor، تنظیمات هر محیط
مید	تداخل نسخه‌ها، BOM و platform، constraint و rich version، taskهای سفارشی lazy
سینیور	قواعد keep در R8، اندازه‌گیری کارایی build، build cache ریموت
آرکیتکت	استراتژی ارتقای AGP، حاکمیت وابستگی‌ها، Variant API در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

مدل ذهنی: اعلام، انتخاب نسخه، بسته‌بندی



- **اعلام‌شده** فقط یک درخواست است: نسخه‌های catalog، BOMها و constraintها همه ورودی فرایند resolution هستند
- **resolve‌شده** حقیقت است: یک نسخه برای هر ماژول روی هر classpath (مثل `debugRuntimeClasspath` و `releaseRuntimeClasspath`) — به‌طور پیش‌فرض بالاترین نسخهٔ درخواستی
- **بسته‌بندی‌شده**: R8 هر چیزی را که نتواند ثابت کند استفاده می‌شود حذف می‌کند؛ reflection به keep rule نیاز دارد

گراف RESOLVE‌شده را دیباگ کنید، نه فایل BUILD را

دستور `./gradlew :app:dependencies` نشان می‌دهد واقعاً چه چیزی ship می‌شود. فایل build فقط نشان می‌دهد چه چیزی درخواست کرده‌اید.



اسماعیل مرادی
@esmaeeil-moradi700

جونیور

مقایسهٔ api و implementation، build type و flavor، تنظیمات هر محیط



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. `api` در برابر `implementation`

configuration	classpath کامپایل خودتان	classpath کامپایل مصرف‌کننده‌ها	زمان اجرا (داخل APK)
`implementation`	بله	—	بله
`api`	بله	بله	بله
`compileOnly`	بله	—	—
`runtimeOnly`	—	—	بله

```
// :core:network — Retrofit types appear in its public API
dependencies {
    api(libs.retrofit)                  // consumers see it
    implementation(libs.okhttp.logging) // internal detail
}
```

- `implementation` وابستگی را از classpath کامپایل مصرف‌کننده‌ها دور نگه می‌دارد — وقتی عوض شود، آن‌ها دوباره کامپایل نمی‌شوند
- `api` را فقط وقتی استفاده کنید که نوع‌های آن وابستگی در API عمومی **خود شما** ظاهر می‌شوند: پارامترها، نوع خروجی، supertypeها، propertyهای عمومی



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱. چیزهایی که مصاحبه‌گر چک می‌کند

تلهٔ مصاحبه

«`implementation` *کتابخانه را از اپ بیرون نگه می‌دارد.*» — نه. کتابخانه همچنان روی classpath زمان اجرا و داخل APK است. فقط تعیین می‌کند چه کسی می‌تواند با آن **کامپایل** کند.

تلهٔ مصاحبه

«*همه‌جا* `api` *بگذار تا چیزی نشکند.*» — آن‌وقت هر ماژول همه چیز را می‌بیند: buildها خیلی بیشتر از لازم دوباره کامپایل می‌شوند و ماژول‌ها ناخواسته به جزئیات داخلی هم وابسته می‌شوند.

سؤال تکمیلی

«*وابستگی‌هایی را که اشتباه اعلام شده‌اند چطور پیدا می‌کنید؟*» — Dependency Analysis Gradle Plugin با دستور `./gradlew buildHealth` وابستگی‌های بلااستفاده، `api` هایی که باید `implementation` باشند، و وابستگی‌های transitive را که بدون اعلام استفاده می‌کنید گزارش می‌دهد.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. build type، product flavor و variant

```kotlin
android {
    flavorDimensions += listOf("tier", "env")
    productFlavors {
        create("free") { dimension = "tier" }
        create("paid") { dimension = "tier" }
        create("dev") {
            dimension = "env"
            applicationIdSuffix = ".dev"
        }
        create("prod") { dimension = "env" }
    }
}
```

- **build type** یعنی *چطور* ساخته می‌شود: قابلیت debug، R8 و امضا؛ مثل `debug` و `release` و شاید `benchmark`
- **product flavor** یعنی چه چیزی ساخته می‌شود: فیچرها، endpointها، برندینگ — در قالب dimensionها
- **variant** یعنی ترکیب یک flavor از هر dimension با یک build type؛ اینجا ۲ × ۲ × ۲ یعنی **هشت** ترکیب، از `freeDevDebug` تا `paidProdRelease`
- source setها ادغام می‌شوند و خاص‌ترین برنده است: اول variant، بعد build type، بعد flavor، و در آخر `main`



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۲. build type مربوط به release

```
buildTypes.getByName("release") {
    isMinifyEnabled = true   // default: false
    isShrinkResources = true
    proguardFiles(
        getDefaultProguardFile("proguard-android-optimize.txt"),
        "proguard-rules.pro",
    )
}
```

تلهٔ مصاحبه

«*buildهای release به‌طور پیش‌فرض با R8 مبهم‌سازی می‌شوند.*» — نه: `isMinifyEnabled` به‌طور پیش‌فرض `false` است، حتی برای `release`. خودتان R8 را روشن می‌کنید — و بعد build مینیفای‌شده را تست می‌کنید. (نسخهٔ پارسال همین چالش خلاف این را گفته بود.)

تلهٔ مصاحبه

«*build type دیباگ LeakCanary را همراه دارد.*» — build typeها کتابخانه نمی‌آورند. شما آن را با `debugImplementation(…)` اضافه می‌کنید، برای همین هرگز به release نمی‌رسد.

سؤال تکمیلی

«*یک ماژول library، build type به اسم `staging` ندارد. چه می‌شود؟*» — variant مربوط به `staging` در اپ نمی‌تواند match شود. به build type `staging` در اپ، `matchingFallbacks += listOf("release")` را اضافه کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. تنظیمات هر محیط

```
android {
    buildFeatures {
        buildConfig = true   // off by default since AGP 8.0
        resValues = true     // off by default since AGP 9.0
    }
    productFlavors {
        create("dev") {
            dimension = "env"
            buildConfigField(
                "String", "API_URL", "\"https://dev.example.com\"",
            )
            resValue("string", "app_name", "Shop Dev")
        }
    }
}
```

- `BuildConfig.API_URL` در کد و `@string/app_name` در manifest استفاده می‌شوند؛ با `applicationIdSuffix`، نسخهٔ dev و prod کنار هم نصب می‌شوند
- مقادیری که باید بدون release عوض شوند (feature switch، kill switch) جایشان در remote config است، نه در flavorها — روز ۲۴



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۳. جایی که secretها *نباید* بروند

تلهٔ مصاحبه

«*کلیدهای API را از کد بیرون نگه دار: در* `local.properties` *بگذار و با* `BuildConfig` *تزریق کن.*» — این کار آن‌ها را از **git** بیرون نگه می‌دارد، نه از **APK**. هر چیزی در `BuildConfig`، resourceها یا کد native با jadx یا apktool قابل استخراج است. secretهای واقعی روی سرور می‌مانند؛ اپ توکن‌های کوتاه‌عمر می‌گیرد — روز ۱۲.

سؤال تکمیلی

«*چرا* `BuildConfig` *حالا به‌طور پیش‌فرض خاموش است؟*» — ساختنش در همهٔ ماژول‌ها زمان build می‌گیرد، و تغییر یک field آن ماژول را دوباره کامپایل می‌کند. فقط در ماژول‌هایی که از آن می‌خوانند روشنش کنید.

FLAVOR یا تنظیمات زمان اجرا؟

flavorها برای چیزهایی هستند که باید **در خود باینری** فرق کنند: application ID، امضا، SDKهای همراه، نسخه‌های مخصوص هر store. بقیه با تنظیمات زمان اجرا ارزان‌تر است.



اسماعیل مرادی
@esmaeeil-moradi700

میدِ

تداخل نسخه‌ها، BOM و platform، constraint و rich version، taskهای سفارشی lazy



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. Gradle چطور نسخه را انتخاب می‌کند

اپ شما OkHttp 4.9.3 را اعلام کرده؛ یک SDK که استفاده می‌کنید به 4.12.0 نیاز دارد. هر classpath فقط یکی را می‌تواند داشته باشد — کدام برنده است؟

- Gradle به‌طور پیش‌فرض **بالاترین** نسخهٔ درخواستی را انتخاب می‌کند («ارتقای خوش‌بینانه») — نه قاعدهٔ نزدیک‌ترین در Maven
- resolution **برای هر classpath جداگانه** انجام می‌شود: `debugRuntimeClasspath` و `releaseRuntimeClasspath` می‌توانند فرق کنند
- platformها (سؤال ۵)، constraintها و rich versionها (سؤال ۶) در همین فرایند شرکت می‌کنند

```
./gradlew :app:dependencies \
    --configuration releaseRuntimeClasspath
```

```
+--- com.example:payments-sdk:3.2.0
|    \--- com.squareup.okhttp3:okhttp:4.12.0
\--- com.squareup.okhttp3:okhttp:4.9.3 -> 4.12.0 (*)
```

علامت `->` یعنی conflict resolution نسخهٔ درخواستی شما را عوض کرده است؛ `(*)` یعنی آن زیرشاخه قبلاً بالاتر نشان داده شده.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۴. پرسیدن *چرا* از Gradle

```
./gradlew :app:dependencyInsight \
    --dependency okhttp \
    --configuration releaseRuntimeClasspath
```

دلیل انتخاب را توضیح می‌دهد: conflict resolution، یک constraint، یک platform، یک نسخهٔ strict، یا یک substitution.

تلهٔ مصاحبه

«نسخه‌ای که در فایل build نوشته‌ام همان نسخه‌ای است که ship می‌شود.» — فقط اگر هیچ چیز نسخهٔ بالاتری نخواهد. یک ارتقای transitive می‌تواند نسخه‌ای ناسازگار در سطح باینری بیاورد و شما آن را با یک `NoSuchMethodError` در زمان اجرا کشف می‌کنید.

سؤال تکمیلی

«ارتقاهای غافلگیرکننده را در code review چطور می‌گیرید؟» — گراف resolveشده را بخشی از diff کنید: dependency locking با فایل `gradle.lockfile`، یا ابزاری مثل Dependency Guard از Dropbox.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. BOM و platform

```
dependencies {
    val composeBom = platform(libs.androidx.compose.bom)
    implementation(composeBom)
    androidTestImplementation(composeBom)

    // No versions: the BOM supplies them
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui.tooling.preview)

    implementation(platform(libs.firebase.bom))
    implementation(libs.firebase.messaging)
}
```

- BOM فهرست منتشرشده‌ای از نسخه‌هاست. `platform()` آن‌ها را به شکل **پیشنهاد** وارد می‌کند که مثل هر نسخهٔ درخواستی دیگری در conflict resolution شرکت می‌کنند
- کتابخانه‌هایی که با هم منتشر می‌شوند هم‌تراز می‌مانند: همهٔ artifactهای Compose، همهٔ SDKهای Firebase
- در catalog، ورودی‌هایی که BOM نسخه‌شان را تعیین می‌کند به‌سادگی `version` ندارند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۵. کاری که BOM انجام نمی‌دهد

تلهٔ مصاحبه

«*اضافه کردن BOM کتابخانه‌ها را اضافه می‌کند.*» — BOM **هیچ** وابستگی‌ای اضافه نمی‌کند — فقط نسخه‌ها را برای آن‌هایی که خودتان اعلام می‌کنید.

تلهٔ مصاحبه

«*نسخهٔ صریح، BOM را override می‌کند.*» — فقط اگر **بالاتر** باشد: بالاترین نسخه برنده است. پایین آوردن نسخه به `strictly` نیاز دارد — و این نشانهٔ یک مشکل است.

تلهٔ مصاحبه

«*برای اطمینان از `enforcedPlatform()` استفاده کن.*» — همهٔ نسخه‌های گراف را override می‌کند، حتی نسخه‌هایی که کتابخانه‌های دیگر لازم دارند. در یک library منتشرشده، انتخاب‌های شما را به همهٔ مصرف‌کننده‌ها تحمیل می‌کند. `platform()` را ترجیح دهید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. constraint، rich version و exclude

ابزار	چه می‌کند	کاربرد
`constraints { }`	*اگر* ماژول در گراف باشد حداقل نسخه را تعیین می‌کند؛ وابستگی اضافه نمی‌کند	وصلهٔ یک وابستگی transitive آسیب‌پذیر
`strictly` یا `1.2.3!!`	هیچ چیز دیگری نمی‌تواند این نسخه را عوض کند — حتی می‌تواند پایین بیاورد	راه فرارهای نادر و مستند
`reject(…)`	نسخه‌های مشخصی را رد می‌کند	یک نسخهٔ منتشرشدهٔ خراب
`exclude(…)`	یک ماژول transitive را حذف می‌کند	کلاس‌های تکراری — وقتی مطمئنید استفاده نمی‌شود
dependency substitution	یک ماژول را با ماژول دیگر یا پروژهٔ محلی عوض می‌کند	forkها، composite buildها

تلهٔ مصاحبه

«`exclude` *مشکل کلاس‌های تکراری را حل می‌کند.*» — خطای build را حل می‌کند و اگر کد هنوز به آن ماژول نیاز داشته باشد، می‌تواند `ClassNotFoundException` در زمان اجرا بسازد. اول ببینید چه کسی از آن استفاده می‌کند (`dependencyInsight`).



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۶. در کد

```
dependencies {
    constraints {
        implementation("com.example:json-parser:2.4.1") {
            because("versions before 2.4.1 have a known CVE")
        }
    }
    implementation("com.example:analytics:5.0.0") {
        exclude(group = "com.example", module = "old-logger")
    }
    implementation("com.example:image-loader") {
        version {
            require("3.1.0")
            reject("3.2.0")   // crashes on API 26
        }
    }
}
```

- متن `because(…)` در خروجی `dependencyInsight` دیده می‌شود — نفر بعدی می‌داند این قاعده چرا وجود دارد
- هر pin یک مالک و یک برنامهٔ خروج لازم دارد؛ pinهای فراموش‌شده سال‌ها جلوی ارتقا را می‌گیرند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. taskهای سفارشی، به روش lazy

```kotlin
abstract class ReleaseNotesTask : DefaultTask() {
    @get:Input
    abstract val versionName: Property<String>

    @get:InputFile
    @get:PathSensitive(PathSensitivity.RELATIVE)
    abstract val changelog: RegularFileProperty

    @get:OutputFile
    abstract val output: RegularFileProperty

    @TaskAction
    fun generate() {
        val text = changelog.get().asFile.readText()
        output.get().asFile
            .writeText("v${versionName.get()}\n$text")
    }
}
```

- propertyهای `abstract` : خود Gradle نمونه‌های `Property` ، `RegularFileProperty` ، `DirectoryProperty` و `ListProperty` را می‌سازد
- ورودی‌ها و خروجی‌های اعلام‌شده بررسی up-to-date را ممکن می‌کنند؛ اگر خروجی ارزش cache شدن دارد، `@CacheableTask` را اضافه کنید



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۷. ثبت task و اتصال خروجی‌ها

```kotlin
val notes = tasks.register<ReleaseNotesTask>("releaseNotes") {
    versionName.set(providers.gradleProperty("appVersion"))
    changelog.set(layout.projectDirectory.file("CHANGELOG.md"))
    output.set(layout.buildDirectory.file("notes/release.txt"))
}

tasks.register<Zip>("bundleNotes") {
    // Wiring the output also wires the task dependency
    from(notes.flatMap { it.output })
    destinationDirectory.set(layout.buildDirectory.dir("dist"))
}
```

تلهٔ مصاحبه

نسخهٔ پارسال یک اسکریپت را با `project.exec {}` داخل `doLast` اجرا می‌کرد. `Project.exec` در Gradle 8.11 منسوخ شد و **در Gradle 9 حذف شد**، و دست زدن به `project` در زمان execution هم configuration cache را خراب می‌کند. از نوع task به اسم `Exec` استفاده کنید، `ExecOperations` را inject کنید، یا برای مقدارها از `providers.exec {}` کمک بگیرید.

تلهٔ مصاحبه

`dependsOn("taskA")` همراه با مسیر hard-codeشده به خروجی آن. به‌جایش `Provider` خروجی را وصل کنید: وابستگی خودکار استنتاج می‌شود و مسیر می‌تواند عوض شود بدون اینکه چیزی بشکند.



اسماعیل مرادی
@esmaeeil-moradi700

سینیور

قواعد keep در R8، اندازه‌گیری کارایی build، build cache ریموت



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. قواعد keep در R8 از کجا می‌آیند

منبع	روی چه چیزی اثر دارد
فایل `proguard-android-optimize.txt` که AGP می‌دهد	قواعد پایهٔ اندروید
فایل `proguard-rules.pro` اپ شما	اپ شما
`consumerProguardFiles(…)` یک library — داخل AAR منتشر می‌شود	همهٔ اپ‌هایی که از آن library استفاده می‌کنند
قواعدی که AAPT2 تولید می‌کند	کلاس‌هایی که در manifest و layoutهای XML به آن‌ها اشاره شده

```
# Fields read by reflection (e.g. Gson DTOs)
-keepclassmembers class com.shop.data.dto.** {
    <fields>;
}
# Class created by name, including its constructor
-keep class com.shop.plugins.PdfRenderer {
    <init>();
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۸. گزینه‌های keep و تغییرات AGP 9

قاعده	اثر
`-keep`	کلاس و اعضای ذکرشده باقی می‌مانند، بدون تغییر نام
`-keepclassmembers`	اعضای ذکرشده باقی می‌مانند — *اگر* خود کلاس به دلیل دیگری باقی بماند
`-keepnames`	اگر استفاده نشود همچنان حذف می‌شود؛ اگر بماند، تغییر نام نمی‌دهد
`,allowobfuscation`	باقی می‌ماند، اما ممکن است تغییر نام بدهد

AGP 9

قاعدهٔ `-keep class A` **دیگر** constructor پیش‌فرض را نگه نمی‌دارد — بنویسید `-keep class A { <init>(); }`. گزینه‌های سراسری مثل `-dontobfuscate` در consumer rules یک library حالا انتشار library را fail می‌کنند، و اپ‌ها آن‌ها را در AARهای از پیش ساخته‌شده نادیده می‌گیرند. `getDefaultProguardFile()` فقط `proguard-android-optimize.txt` را قبول می‌کند.

تلهٔ مصاحبه

قاعدهٔ `-keep class com.shop.** { *; }` «برای اطمینان»، shrink و مبهم‌سازی را برای کل کد شما خاموش می‌کند. محدود keep کنید، تولید کد (KSP، kotlinx.serialization) را به reflection ترجیح دهید، و تست‌های UI را روی build مینیفای‌شده اجرا کنید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. اندازه‌گیری کارایی build

ابزار	به چه سؤالی جواب می‌دهد
build scan با `--scan`	timeline، زمان configuration، hit و miss در cache، دانلود وابستگی‌ها
Build Analyzer در Android Studio	taskهایی که بیشترین زمان را می‌گیرند، سازگاری با configuration cache، اینکه Jetifier هنوز لازم است یا نه
Gradle Profiler	benchmark تکرارپذیر از سناریوهای واقعی، با warm-up
پرچم `--profile`	یک گزارش HTML سادهٔ محلی در `build/reports/profile`

```
# performance.scenarios
cart_change {
    tasks = [":app:assembleDebug"]
    apply-non-abi-change-to = "cart/src/main/kotlin/Cart.kt"
    warm-ups = 3
    iterations = 10
}
```

```
gradle-profiler --benchmark --project-dir . \
    --scenario-file performance.scenarios cart_change
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۹. اول به چه چیزی نگاه کنیم

- **زمان configuration** — در هر build بدون cache hit و در هر sync پرداخت می‌شود
- **cache miss** در taskهایی که باید cacheable باشند: مسیرهای مطلق، timestampها، JDKهای متفاوت
- **kapt**، فلگ `android.enableJetifier`، نسخه‌های dynamic و زمان GC
- یک ماژول غول‌پیکر `:core` که همه به آن وابسته‌اند: هر تغییرش همه چیز را دوباره کامپایل می‌کند

تلهٔ مصاحبه

«*ابزار این کار Gradle Enterprise است.*» — اسمش در ۲۰۲۴ به **Develocity** تغییر کرد. build scanها روی scans.gradle.com رایگان مانده‌اند؛ نسخهٔ سازمانی Develocity امکانات تیمی اضافه می‌کند: سرور اختصاصی، cache ریموت و تحلیل در سطح تیم.

تلهٔ مصاحبه

طراحی build analytics در نسخهٔ پارسال از `BuildListener` و `TaskExecutionListener` استفاده می‌کرد. هر دو منسوخ‌اند و با configuration cache کار نمی‌کنند. از Develocity استفاده کنید، یا از یک `BuildService` مشترک که `OperationCompletionListener` را پیاده می‌کند و از طریق `BuildEventsListenerRegistry` ثبت می‌شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. build cache ریموت

```kotlin
// settings.gradle.kts
val isCi = providers.environmentVariable("CI").isPresent
fun env(name: String) = providers.environmentVariable(name).orNull

buildCache {
    local { isEnabled = !isCi }
    remote<HttpBuildCache> {
        url = uri("https://cache.example.com/cache/")
        isPush = isCi
        credentials {
            username = env("CACHE_USER")
            password = env("CACHE_PASSWORD")
        }
    }
}
```

- **CI می‌نویسد، توسعه‌دهنده‌ها می‌خوانند.** ماشین یک توسعه‌دهنده نمی‌تواند cache مشترک را آلوده کند
- backendها: build cache node در Develocity، یا هر سرور HTTP که پروتکل `GET` و `PUT` مربوط به Gradle را پیاده کند
- خود caching را با `org.gradle.caching=true` روشن کنید (روز ۲۱)



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۰. وقتی cache ریموت کمکی نمی‌کند

تلهٔ مصاحبه

نسخهٔ پارسال آن را با `org.gradle.caching.remote.enabled=true` فعال می‌کرد. چنین propertyای وجود ندارد: cache ریموت در `settings.gradle.kts` و داخل بلوک `buildCache {}` تنظیم می‌شود.

تلهٔ مصاحبه

«*cache ریموت اضافه کردیم، پس CI سریع است.*» — فقط اگر کلیدها match شوند. مسیرهای مطلق در ورودی‌ها، vendor متفاوت JDK یا line ending سیستم‌عامل، و timestampهای تولیدشده هر task را به miss تبدیل می‌کنند. build scan نشان می‌دهد *کدام ورودی* بین دو build فرق داشته است.

سؤال تکمیلی

«*ممکن است cache ریموت build را کندتر کند؟*» — بله. دانلود خروجی هزینه دارد؛ برای taskهای ارزان روی شبکهٔ کند، ساختن محلی برنده است. نرخ hit **و** زمان صرفه‌جویی‌شده را اندازه بگیرید.



اسماعیل مرادی
@esmaeeil-moradi700

آرکیتکت

استراتژی ارتقای AGP، حاکمیت وابستگی‌ها، Variant API در مقیاس بزرگ



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. یک استراتژی برای ارتقای AGP

جزء	به چه چیزی وابسته است
Android Studio	هر نسخه بازه‌ای از نسخه‌های AGP را پشتیبانی می‌کند
AGP	حداقل نسخهٔ Gradle؛ برای اجرا JDK 17 یا بالاتر (AGP 9.0 به Gradle 9.1 یا بالاتر نیاز دارد)
پلاگین Kotlin برای Gradle	بازه‌های تست‌شدهٔ AGP و Gradle؛ AGP 9 در زمان اجرا به KGP وابسته است
KSP، Hilt و پلاگین‌های دیگر	باید نسخه‌های AGP، Kotlin و Gradle شما را پشتیبانی کنند

۱. اول release noteها و جدول‌های سازگاری را بخوانید
۲. هشدارهای deprecation را روی نسخهٔ **فعلی** رفع کنید (با `--warning-mode all`) — نسخهٔ major بعدی آن APIها را حذف می‌کند
۳. **AGP Upgrade Assistant** را اجرا کنید؛ در هر PR فقط یک ابزار را ارتقا دهید
۴. نسخه‌های preview را روی یک branch بسازید (canary در Android Studio، نسخهٔ alpha یا beta از AGP) تا شکستگی‌ها زود پیدا شوند
۵. تأیید: مقایسهٔ build scanها، diff خروجی APK یا AAB (حجم، manifest، DEX)، و تست‌های UI روی build مینیفای‌شده



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۱. به‌طور خاص AGP 9

AGP 9

- **Kotlin داخلی** به‌طور پیش‌فرض روشن است: `org.jetbrains.kotlin.android` را حذف کنید؛ `kotlin-kapt` باید به KSP یا `com.android.legacy-kapt` منتقل شود
- **فقط DSL جدید**: نوع‌های قدیمی به سبک `BaseExtension` و variant API قدیمی (`applicationVariants`) پنهان شده‌اند
- پرچم‌های انصراف (`android.newDsl=false` و `android.builtInKotlin=false`) پل‌های موقت‌اند — قرار است در AGP 10 حذف شوند

تلهٔ مصاحبه

«*تا چیزی مجبورمان نکند روی AGP قدیمی می‌مانیم.*» — الزام targetSdk در Play هر سال بالا می‌رود، compileSdk جدید AGP جدید می‌خواهد، و رد کردن چند نسخهٔ major یک ارتقای معمولی را به یک پروژه تبدیل می‌کند. ارتقاهای کوچک و منظم ارزان‌ترند.

تلهٔ مصاحبه

«*با پرچم‌های انصراف، کار AGP 9 تمام است.*» — این پرچم‌ها فقط مهاجرت را عقب می‌اندازند. هر پلاگین third-party که هنوز به آن‌ها نیاز دارد را پیگیری کنید و برای حذفشان برنامه بریزید.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. حاکمیت وابستگی‌ها در مقیاس بزرگ

مشکل	کنترل
نسخه‌ها بین ماژول‌ها و ریپوها از هم فاصله می‌گیرند	یک catalog، به‌علاوهٔ یک **platform** داخلی (BOM) که همهٔ اپ‌ها import می‌کنند
وابستگی‌های اشتباه اعلام‌شده یا بلااستفاده	Dependency Analysis Gradle Plugin با `buildHealth` در CI
ارتقاهای غافلگیرکننده	dependency locking یا چک diff وابستگی‌ها در هر PR
حملات زنجیرهٔ تأمین، dependency confusion	تأیید وابستگی‌ها؛ `exclusiveContent` برای groupهای داخلی
کتابخانه‌های کهنه	Renovate یا Dependabot با PRهای کوچک و مکرر

تلهٔ مصاحبه

«*اضافه کردن یک کتابخانه تغییر یک‌خطی است.*» — کل گراف transitive آن را اضافه می‌کند: حجم APK، keep ruleها، لایسنس‌ها، و شاید ارتقای بی‌صدای نسخه‌هایی که الان استفاده می‌کنید. وابستگی third-party جدید باید review شود.



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۲. یک platform داخلی و فیلتر کردن مخزن‌ها

```
// platform/build.gradle.kts — the company BOM
plugins { `java-platform` }
javaPlatform { allowDependencies() }

dependencies {
    api(platform(libs.androidx.compose.bom))
    constraints {
        api(libs.okhttp)
        api(libs.kotlinx.coroutines.core)
    }
}
```

```
// settings.gradle.kts — internal groups only from our repo
dependencyResolutionManagement.repositories {
    exclusiveContent {
        forRepository { maven("https://maven.example.com") }
        filter { includeGroupByRegex("com\\.shop(\\..*)?") }
    }
}
```



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. انفجار variantها و Variant API

دو dimension با سه flavor و سه build type یعنی همین حالا ۲۷ variant. هر کدام زمان configuration، زمان sync و سطح CI را بیشتر می‌کند.

```
androidComponents {
    beforeVariants { variant ->
        // Nobody ships or tests the dev backend in release
        if (variant.buildType == "release" &&
            variant.productFlavors.contains("env" to "dev")
        ) {
            variant.enable = false
        }
    }
}
```

- variantهای غیرفعال اصلاً task نمی‌گیرند — ارزان‌تر از نادیده گرفتنشان
- تنظیمات زمان اجرا را به dimension جدید ترجیح دهید؛ برای تعداد زیادی برند white-label، ماژول‌های اپ جدا که ماژول‌های فیچر را به اشتراک می‌گذارند بهترند



اسماعیل مرادی
@esmaeeil-moradi700

سؤال ۱۳. task سفارشی برای هر variant

```
androidComponents {
    onVariants(selector().withBuildType("release")) { variant ->
        tasks.register<ApkSizeTask>("${variant.name}ApkSize") {
            apkDir.set(variant.artifacts.get(SingleArtifact.APK))
            loader.set(variant.artifacts.getBuiltArtifactsLoader())
        }
    }
}
```

- `SingleArtifact.APK` یک `Provider` تنبل است: وصل کردنش وابستگی به task بسته‌بندی را خودکار اضافه می‌کند
- داخل task: یک `@get:InputFiles abstract val apkDir:` `DirectoryProperty` و یک `BuiltArtifactsLoader` با `@Internal` برای فهرست کردن APKها

تلهٔ مصاحبه

`applicationVariants.all { }` همراه با `dependsOn("assembleRelease")` و مسیر hard-codeشدهٔ `build/outputs/apk/…` . variant API قدیمی در AGP 9 به‌طور پیش‌فرض غیرفعال است و مسیر خروجی‌ها جزئیات پیاده‌سازی است. از `androidComponents` و artifacts API استفاده کنید.

جواب سینیور در یک جمله

«محدود اعلام کن، گراف resolveشده را بررسی کن، خروجی‌ها را به‌جای مسیرها وصل کن — و در قدم‌های کوچک و اندازه‌گیری‌شده ارتقا بده.»



اسماعیل مرادی
@esmaeeil-moradi700

روز ۲۲ — خلاصهٔ یک‌صفحه‌ای

- پیش‌فرض `implementation`؛ `api` فقط برای نوع‌های API عمومی.
- variant یعنی یک flavor از هر dimension ضرب در build type.
- R8 **خاموش** است مگر با `isMinifyEnabled = true`. build مینیفای‌شده را تست کنید.
- `BuildConfig` و resourceها جای نگهداری secret نیستند.
- Gradle **بالاترین** نسخهٔ درخواستی را انتخاب می‌کند؛ با `dependencyInsight` بررسی کنید.
- BOM نسخه اضافه می‌کند، نه کتابخانه. `platform()` را به `enforcedPlatform()` ترجیح دهید.
- taskهای سفارشی: ورودی `Property`، خروجی annotateشده، `register`، و اتصال `Provider`ها. بدون `project` در action.
- keep ruleها: محدود و بر اساس reflection. در AGP 9، `-keep class A` دیگر constructor را نگه نمی‌دارد.
- اندازه‌گیری با build scan و Gradle Profiler؛ cache ریموت: CI می‌نویسد، توسعه‌دهنده‌ها می‌خوانند.
- ارتقا: اول deprecationها، هر PR یک ابزار. AGP 9 یعنی Kotlin داخلی، DSL جدید و `androidComponents`.

روز ۲۳ در راه است: CI/CD بخش ۱ — pipelineها، caching و تحلیل ایستا. این پست را ذخیره کنید و همراه ما باشید.



اسماعیل مرادی
@esmaeeil-moradi700